EXPT NO:07	ANALYSIS OF BAYESIAN DECISION THEORY FOR CONTINUOUS FEATURES
DATE:20.02.2026	

AIM:

To analyse bayesian decision theory for continuous feature to classify Tumor vs normal tissue in brain MRI images.

PROGRAM CODE:

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d navoneel/brain-mri-images-for-brain-tumor-detection
!unzip brain-mri-images-for-brain-tumor-detection.zip
!ls
dataset_path = "/content/brain_tumor_dataset"
features = []
labels = []
kernel_size = 5
for label, folder in enumerate(['no', 'yes']):   # no=0, yes=1
    folder_path = os.path.join(dataset_path, folder)
    for img_name in os.listdir(folder_path):
        img_path = os.path.join(folder_path, img_name)
        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        image = cv2.resize(image, (128,128))
        # Compute local variance
        mean = cv2.blur(image.astype(np.float32), (kernel_size, kernel_size))
        mean_sq = cv2.blur((image.astype(np.float32))**2, (kernel_size, kernel_size))
        variance = mean_sq - mean**2
        # Take average intensity & average variance for whole image
```

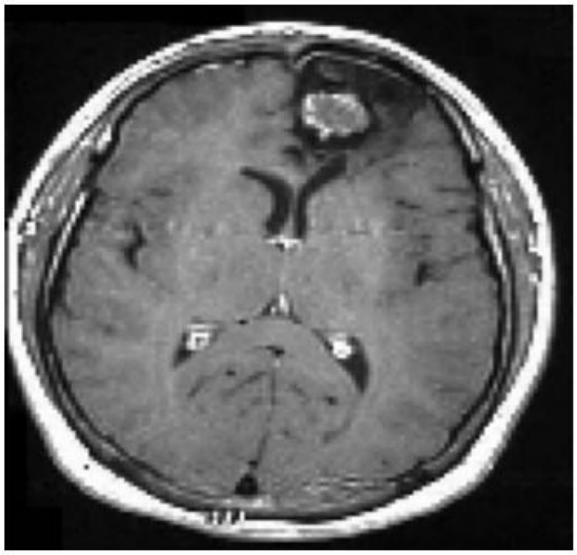
```
        avg_intensity = np.mean(image)
        avg_variance = np.mean(variance)
        features.append([avg_intensity, avg_variance])
        labels.append(label)
features = np.array(features)
labels = np.array(labels)
# Separate classes
normal_features = features[labels == 0]
tumor_features = features[labels == 1]
# Compute mean
mu_normal = np.mean(normal_features, axis=0)
mu_tumor = np.mean(tumor_features, axis=0)
# Compute covariance
cov_normal = np.cov(normal_features.T)
cov_tumor = np.cov(tumor_features.T)
# Compute priors
prior_normal = len(normal_features) / len(features)
prior_tumor = len(tumor_features) / len(features)
prior_normal
prior_tumor
def gaussian_pdf(x, mean, cov):
    size = len(x)
    det = np.linalg.det(cov)
    inv = np.linalg.inv(cov)
    norm_const = 1.0 / (np.power((2*np.pi), float(size)/2) * np.sqrt(det))
    x_mu = x - mean
    result = np.exp(-0.5 * np.dot(np.dot(x_mu, inv), x_mu.T))
    return norm_const * result
predictions = []
for x in features:
```

```
    likelihood_normal = gaussian_pdf(x, mu_normal, cov_normal)
    likelihood_tumor = gaussian_pdf(x, mu_tumor, cov_tumor)
    posterior_normal = likelihood_normal * prior_normal
    posterior_tumor = likelihood_tumor * prior_tumor
    if posterior_tumor > posterior_normal:
        predictions.append(1)
    else:
        predictions.append(0)
predictions = np.array(predictions)
accuracy = np.sum(predictions == labels) / len(labels)
print("Accuracy:", accuracy)
prior_tumor = 0.5
prior_normal = 0.5
def extract_features_from_image(img_path):
    image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    image = cv2.resize(image, (128,128))
    mean = cv2.blur(image.astype(np.float32), (kernel_size, kernel_size))
    mean_sq = cv2.blur((image.astype(np.float32))**2, (kernel_size, kernel_size))
    variance = mean_sq - mean**2
    avg_intensity = np.mean(image)
    avg_variance = np.mean(variance)
    return np.array([avg_intensity, avg_variance]), image
def predict_image(img_path):
    x, image = extract_features_from_image(img_path)
    likelihood_normal = gaussian_pdf(x, mu_normal, cov_normal)
    likelihood_tumor = gaussian_pdf(x, mu_tumor, cov_tumor)
    posterior_normal = likelihood_normal * prior_normal
    posterior_tumor = likelihood_tumor * prior_tumor
    if posterior_tumor > posterior_normal:
        prediction = "Tumor"
    else:
        prediction = "Normal"
    # Show image
    plt.imshow(image, cmap='gray')
    plt.title(f"Prediction: {predition}")
    plt.axis('off')
    plt.show()
    return prediction
```

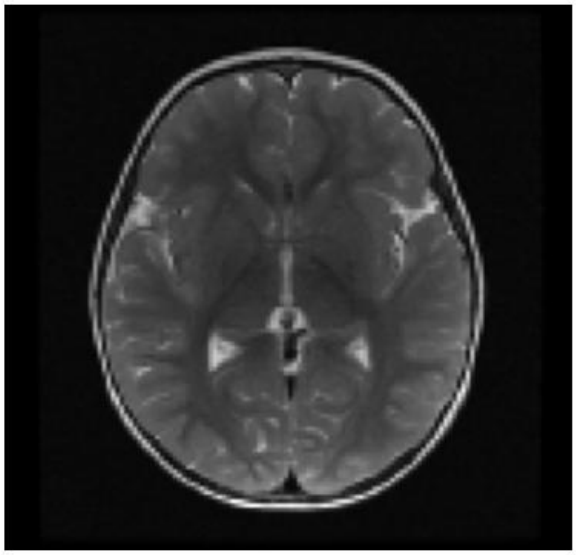
```
test_image_path = "/content/brain_tumor_dataset/yes/Y12.jpg"
predict_image(test_image_path)
test_image_path = "/content/brain_tumor_dataset/no/2 no.jpeg"
predict_image(test_image_path)
def segmentation_map(img_path):
    image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    image = cv2.resize(image, (128,128))
    # Compute local variance
    mean = cv2.blur(image.astype(np.float32), (kernel_size, kernel_size))
    mean_sq = cv2.blur((image.astype(np.float32))**2, (kernel_size, kernel_size))
    variance = mean_sq - mean**2
    segmentation = np.zeros(image.shape)
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            x = np.array([
                image[i,j],
                variance[i,j]
            ])
            likelihood_normal = gaussian_pdf(x, mu_normal, cov_normal)
            likelihood_tumor = gaussian_pdf(x, mu_tumor, cov_tumor)
            posterior_normal = likelihood_normal * prior_normal
            posterior_tumor = likelihood_tumor * prior_tumor

            if posterior_tumor > posterior_normal:
                segmentation[i,j] = 1
            else:
                segmentation[i,j] = 0
    # Show results
    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1)
    plt.imshow(image,cmap='gray')
    plt.title("Original MRI")
    plt.axis('off')
    plt.subplot(1,2,2)
    plt.imshow(segmentation,cmap='gray')
    plt.title("Segmentation Map")
    plt.axis('off')
    plt.show()
segmentation_map("/content/brain_tumor_dataset/yes/Y12.jpg")
segmentation_map("/content/brain_tumor_dataset/no/2 no.jpeg")
```

OUTPUT:

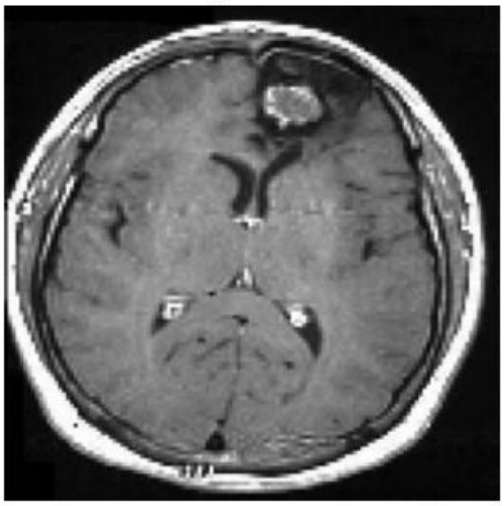
Prediction: Tumor



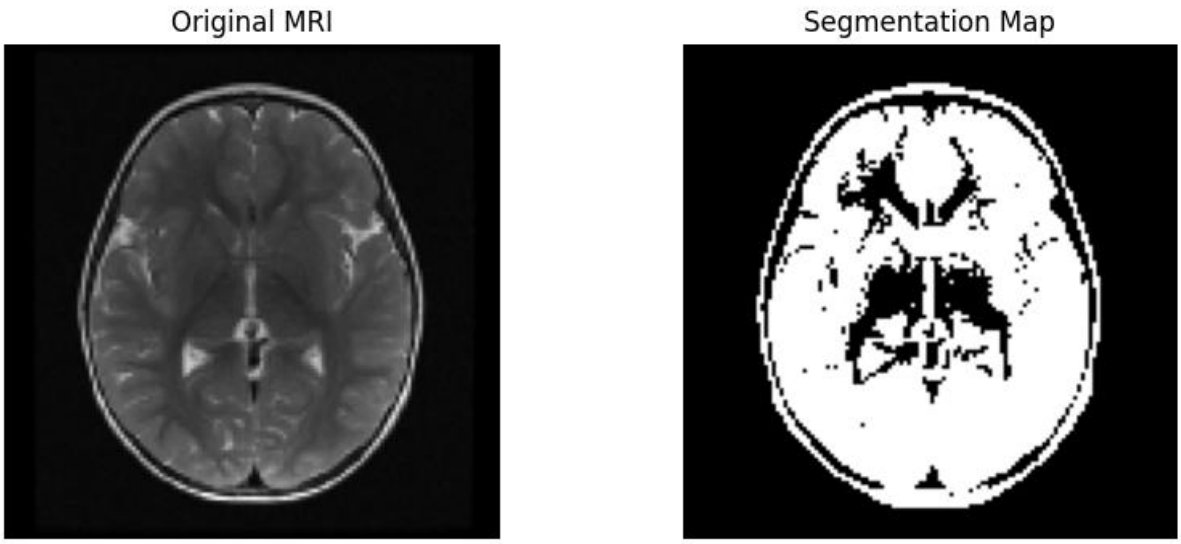
Prediction: Normal



Original MRI



Segmentation Map



RESULT:

Thus, the analysis of bayesian decision theory for continuous feature to classify Tumor vs normal tissue in brain MRI images has been done successfully.

2303717624322023